

GRAPHIC ERA (DEEMED TO BE UNIVERSITY), DEHRADUN

Department of Computer Science and Engineering

PRACTICE SHEET

Stack Applications: Expression Conversion and Evaluation

Course: Data Structures Using C

Subject Code: TCS 302

Name: _____

Roll No.: _____

Section: _____

Date: _____

Instructions

- Show intermediate steps wherever applicable.
- Show stack contents for algorithm-based conversions and evaluations.
- Assume the precedence:
$$> *, /, \% > +, -$$
- Assume $\wedge$ is right associative and other binary operators are left associative.

Section A: Basic Concepts

Q1. Define the following terms with one suitable example each:

- (a) Infix Expression
- (b) Prefix Expression
- (c) Postfix Expression

Q2. Complete the following table:

Infix	Prefix	Postfix
$A + B$		
$A - B$		
$A * B$		
A / B		
$(A + B) * C$		

Section B: Infix to Postfix Conversion

Q3. Convert the following infix expressions into postfix expressions:

(a) $A - B + C$

(d) $A + B * C - D$

(b) $A * B + C / D$

(e) $(A + B) * (C - D)$

(c) $(A - B) + C * D$

(f) $A * (B + C) / D$

Q4. Convert the following expressions into postfix notation. For each expression, show the **operator stack after every symbol scanned**.

(a) $(A - B) * (C + D)$

(b) $(A + B) / (C + D) - (D * E)$

(c) $A - (B / C + (D \% E * F) / G) * H$

Q5. Convert the following infix expressions into postfix form:

(a) $(A - B) + C * D / E - C$

(b) $(A * B) + (C / D) - (D + E)$

(c) $((A - B) + D / ((E + F) * G))$

(d) $(A - 2 * (B + C) / D * E) + F$

(e) $14 / 7 * 3 - 4 + 9 / 2$

Section C: Infix to Prefix Conversion

Q6. Convert the following infix expressions into prefix notation:

- (a) $(A + B) * C$
- (b) $(A - B) * (C + D)$
- (c) $(A + B) / (C + D) - (D * E)$
- (d) $A + B * C$
- (e) $(A + B) * (C - D)$

Q7. Convert the following expression into prefix notation and show all major conversion steps:

$$(A - B/C) * (A/K - L)$$

Use the following procedure:

- (a) Reverse the infix expression.
- (b) Interchange opening and closing parentheses.
- (c) Convert the resulting expression into postfix.
- (d) Reverse the postfix expression.

Section D: Postfix to Infix Conversion

Q8. Find the infix equivalents of the following postfix expressions:

- (a) $AB+C*D-$
- (b) $ABC*+D-$
- (c) $AB-CD+*$
- (d) $ABC/+DE*-$
- (e) $AB+CDE+**$

Section E: Prefix to Infix Conversion

Q9. Find the infix expressions corresponding to the following prefix expressions:

- (a) $*-+ABCD$
- (b) $+-A*BCD$
- (c) $+A*BC$
- (d) $*-AB+CD$
- (e) $/-AB+CD$

Section F: Postfix Expression Evaluation

Q10. Evaluate the following postfix expressions. Show the **operand stack after processing every token**.

(a) $23+5*$

(b) $82-3*$

(c) $9\ 3\ 4\ *\ 8\ +\ 4\ /\ -$

(d) $6\ 2\ /\ 3\ -\ 4\ *$

Q11. Convert the following infix expression into postfix notation and then evaluate it:

$$10 + \frac{(7 - 5) + 10}{2}$$

Show:

- The original infix expression
- The corresponding postfix expression
- Operand stack after every operation
- Final result

Q12. Convert the following expression into postfix notation and evaluate it:

$$(8 + 2) * (6 - 3)$$

Q13. Convert the following expression into postfix notation and evaluate it:

$$20/(5 - 1) + 3 * 2$$

Q14. Convert the following expression into postfix notation and evaluate it:

$$(9 - 3) * (4 + 2)/3$$

Section G: Error Detection and Conceptual Analysis

Q15. Determine whether each of the following infix expressions is **valid** or **invalid**. Give a reason for your answer.

(a) $(A + B) * (C - D)$

(b) $(A + B] * (C - D)$

(c) $A + B * (C - D)$

(d) $((A + B) * C)$

(e) $A + ((B * C) - D)$

Q16. A student performs the following conversion:

$$A + B * C$$

and obtains:

$$AB+C*$$

Is the conversion correct?

If not:

- (a) Identify the mistake.
- (b) Explain the precedence rule violated.
- (c) Write the correct postfix expression.

Q17. A student evaluates the postfix expression

$$84-$$

as:

$$4 - 8 = -4$$

Explain the error and determine the correct result.

Q18. Explain why the order of operands is particularly important while evaluating the operators:

$$-, \quad /, \quad \%$$

using a stack.

Section H: Higher-Order Problems

Q19. Convert the following expression into both **postfix** and **prefix** notation:

$$A * (B + C) - D / (E + F)$$

Q20. Convert the following infix expression into postfix notation:

$$A + B * (C^D - E)^{(F+G*H)} - I$$

Show the operator stack during conversion.

Q21. Consider the expressions:

$$A + B * C$$

and

$$(A + B) * C$$

Answer the following:

- (a) Find the postfix form of both expressions.
- (b) Why are their postfix expressions different?
- (c) What role does operator precedence play?

Q22. Explain why a correctly formed postfix expression does not require parentheses during evaluation.

Q23. Explain why postfix expressions are particularly suitable for stack-based evaluation.

Section I: Coding Practice

Q24. Write a C program to convert an infix expression into postfix notation using a stack.

Your program must support:

$$+, \quad -, \quad *, \quad /, \quad \%, \quad (, \quad)$$

The program should contain separate functions for:

- `push()`
- `pop()`
- `peek()`
- `precedence()`
- `infixToPostfix()`

Q25. Write a C program to evaluate a postfix expression using a stack.

Your program must correctly preserve operand order for:

$$-, \quad /, \quad \%$$

Q26. Challenge: Modify the postfix evaluation program to support multi-digit operands.

For example:

$$10 \ 7 \ 5 \ - \ 10 \ + \ 2 \ / \ +$$

instead of assuming that every operand contains only one digit.

Section J: Quick Challenge

5-Minute Challenge

Without writing the complete conversion steps, determine the postfix forms of the following expressions:

1. $A + B * C$
2. $(A + B) * C$
3. $A * (B + C)$
4. $(A - B) / (C + D)$
5. $A - B * C + D / E$

Bonus Question:

Which of the expressions above, if any, produce the same ordering of operators in their postfix representations? Explain your observation.

Section K: Trace-the-Stack Challenge

For the following expression:

$$A - (B / C + (D \% E * F) / G) * H$$

complete the table while converting it to postfix.

Symbol Scanned	Operator Stack	Postfix Expression
A		
-		
(		
B		
/		
C		
+		
(		
D		
%		
E		
*		
F		
)		
/		
G		
)		
*		
H		
END		

Think → Trace → Code → Test

Do not memorize expression conversions. Understand how the stack behaves.